BGG Vault API Technical Report

COMP3011 | Web Services and Web Data
Danial Eizlan Bin Daud | 201908257

GitHub Repository	github.com/deezlan/bgg-vault-api
API Documentation	github.com/deezlan/bgg-vault-api/blob/main/…
Presentation Slides	Canva link

1. Project Overview

BGG Vault API is a data-driven REST API for tracking and analysing personal board game collections, built on top of the BoardGameGeek (BGG) dataset. The core concept is to provide users with a personal collection layer (tracking ownership, play counts, and personal ratings) on top of a pre-seeded catalogue of approximately 20,000 real board games sourced from Kaggle. The API goes beyond basic CRUD by offering analytics endpoints such as collection statistics, mechanic-based game recommendations, and trending games by decade.

The project was motivated by a personal interest in tabletop gaming, particularly strategy and cooperative games. BoardGameGeek is the de facto community database for the hobby, making its dataset both rich and meaningful for this kind of application. The combination of real-world data, authentication, and personalised analytics made this a well-rounded project that satisfies all coursework requirements while remaining genuinely useful.

2. Technology Stack

Choosing the right stack was a deliberate decision driven by the requirements of a pure API project with a tight development timeline.

2.1 Framework — FastAPI

I selected **FastAPI** over Django REST Framework and Node.js Express for three reasons. First, FastAPI auto-generates interactive Swagger UI documentation at `/docs` from type annotations alone, eliminating the need to write documentation separately and ensuring it stays in sync with the code. Second, its use of Pydantic for request and response validation means constraints are declared once in schemas and enforced automatically across all routes. Third, FastAPI is increasingly the industry standard for Python API development due to its async-first architecture and performance characteristics, making it a more forward-looking choice than Django for a pure API project.

2.2 Database — SQLite via SQLAlchemy

I chose **SQLite** as the database because it is a self-contained, file-based SQL database that requires no installation or server configuration which is ideal for a development and demonstration context. The entire database lives in a single `.db` file, which simplifies setup for examiners running the project locally. SQLAlchemy provides a clean ORM layer over SQLite, allowing me to define models as Python classes and avoid raw SQL while still producing well-structured relational schemas. The primary limitation of SQLite is its unsuitability for high-concurrency production environments; a natural upgrade path would be to swap the connection string for PostgreSQL, which SQLAlchemy supports without any model changes.

2.3 Authentication — JWT with passlib and python-jose

User authentication is implemented using **JSON Web Tokens (JWT)** signed with the HS256 algorithm. Passwords are hashed using bcrypt via passlib before storage, meaning plaintext passwords are never persisted. Tokens are issued on login, expire after 30 minutes by default, and must be included as a Bearer token on all protected endpoints. This approach is stateless which is consistent with REST principles and scales more naturally than session-based authentication.

2.4 Configuration — pydantic-settings

Sensitive configuration values (`SECRET_KEY`, `DATABASE_URL`, token expiry) are managed via a `.env` file loaded through pydantic-settings. The `.env` file is excluded from version control via `.gitignore`, and a `.env.example` template is committed in its place so collaborators know what variables are required without exposing real secrets.

3. Architecture and Design

3.1 Project Structure

The project follows a modular router-based architecture, separating concerns across distinct layers. The app/ directory contains the database configuration, SQLAlchemy models, Pydantic schemas, authentication logic, and individual router files. Each router is responsible for a single domain (games, collection, and auth) and is registered in main.py with a dedicated prefix and tag for Swagger grouping.

3.2 Data Models

Three SQLAlchemy models underpin the API: Game, User, and Collection. The Game model is read-only from the user's perspective, seeded from the BoardGameGeek CSV dataset. The User model stores hashed credentials. The Collection model acts as a join between users and games, adding personal metadata such as status, personal rating, play count, and notes. All foreign key relationships are enforced at the database level.

3.3 Schema-Level ValidationS

A deliberate design decision was to push as much validation as possible into the Pydantic schema layer rather than scattering checks throughout route logic. Key constraints include: personal_rating bounded between 1.0 and 10.0 to match BGG's own scale; play_count restricted to non-negative integers; notes capped at 500 characters to prevent database abuse; passwords capped at 72 bytes for bcrypt compatibility; usernames restricted to alphanumeric characters and underscores; and CollectionStatus implemented as a string Enum so Pydantic rejects invalid values automatically and Swagger renders a dropdown selector.

3.4 Endpoint Design

Method	Endpoint	Purpose
GET	/games	Browse with search, filter, pagination
GET	/games/trending	Top rated games, filterable by decade
GET	/games/recommend	Top rated games by mechanic
GET	/games/{id}	Full detail for a single game
POST	/auth/register	Register a new user account
POST	/auth/login	Login and receive JWT access token
GET	/auth/me	Return current authenticated user
GET	/collection	View user's collection (protected)
POST	/collection	Add a game to collection (protected)
PATCH	/collection/{id}	Update collection entry (protected)
DELETE	/collection/{id}	Remove from collection (protected)
GET	/collection/stats	Analytics summary (protected)
GET	/collection/recommend	Personalised recommendations (protected)

4. Dataset

The API is seeded from the **Board Games dataset by andrewmvd** on Kaggle (kaggle.com/datasets/andrewmvd/board-games), which contains approximately 20,000 board games scraped from boardgamegeek.com. The dataset is licensed for academic use and includes fields for game title, year published, player counts, play time, complexity rating, average user rating, mechanics, and domains.

The dataset is not committed to the repository for two reasons: its file size would bloat the Git history, and redistributing it without explicit permission may violate Kaggle's terms of service. Instead, setup instructions in `README.md` direct users to download the file and place it in the `data/` directory before running the seed script.

The seed script (`seed.py`) handles several data quality issues: the CSV uses semicolons as delimiters and commas as decimal separators (European format), both of which require explicit handling. Rows with missing titles or implausible year values (outside 1900–2025) are filtered out. Records are inserted in batches of 500 for performance, and the script is idempotent.

5. Testing and Error Handling

5.1 Test Suite

I wrote a pytest test suite of 63 tests covering all three routers, using an isolated SQLite test database that is created fresh at the start of each test session and torn down on completion. A shared conftest.py file provides fixtures for the test client, database override, seeded test data, and JWT authentication headers. This ensures tests are independent of the production database and repeatable without manual setup.

Test File	Tests
tests/test_auth.py	13
tests/test_games.py	22
tests/test_collection.py	28
Total	63

5.2 Error Handling

Three global exception handlers are registered in main.py. A RequestValidationError handler intercepts Pydantic validation failures and returns structured 422 responses with field-level error messages. An IntegrityError handler catches database constraint violations, such as duplicate email registration, and returns a clean 400 rather than exposing a raw SQLAlchemy traceback. A generic Exception handler acts as a catch-all, ensuring that any unhandled error returns a 500 with a safe message rather than internal stack trace details. This last point is a security consideration: exposing tracebacks to clients can reveal implementation details useful to attackers.

6. Challenges and Lessons Learned

Several practical challenges arose during development that deepened my understanding of both the tools and general API design principles.

The most immediately visible issue was a **UTF-8 BOM prefix (\ufeff)** on the CSV's ID column header, which caused the seed script to fail to locate the column by name. The fix was simply changing the file encoding from utf-8 to utf-8-sig, but it illustrated how real-world datasets carry subtle formatting artefacts that documentation does not mention.

A more instructive challenge involved **unhandled database constraint errors** surfacing as 500 responses. When a user attempted to register with a duplicate email address, SQLAlchemy raised an IntegrityError that propagated to the client as a raw 500. The correct fix was twofold: add an explicit application-layer check that returns a meaningful 400 before the database is even reached, and add a global IntegrityError handler as a fallback for any constraint violation not caught explicitly.

Defining **CollectionStatus as a string Enum** rather than validating status manually in the router also improved the design significantly. The original version used a set of valid strings checked inside the route function, which was duplicated across POST and PATCH. Moving validation to the schema layer meant Pydantic rejected invalid values automatically, Swagger rendered a dropdown selector, and the router code became cleaner.

The bcrypt library raised an error when password hashing was first tested, related to a version incompatibility between passlib and bcrypt. This was resolved by pinning compatible versions in requirements.txt. A related constraint (**bcrypt silently truncating passwords over 72 bytes**) was addressed by adding a max_length=72 constraint at the Pydantic schema level, making the truncation boundary explicit rather than a silent behaviour.

7. Limitations and Future Improvements

The current implementation has several known limitations that would be addressed before any production deployment.

- SQLite is unsuitable for concurrent production traffic. Migrating to PostgreSQL would require only a connection string change in .env, as SQLAlchemy abstracts the underlying database engine.
- The SECRET_KEY is currently a static string. A production system should rotate keys and implement token revocation via a blocklist or refresh token mechanism.
- Mechanics and categories are stored as comma-separated strings in the Game model. Normalising these into separate tables with many-to-many relationships would enable more precise filtering and better query performance.
- The recommendation algorithm is simple: it identifies top mechanics from the user's collection and surfaces highly rated games sharing those mechanics. A more sophisticated approach could use collaborative filtering or vector similarity across mechanic profiles.
- There is no rate limiting on any endpoint. A production API would apply rate limits per IP or per user to prevent abuse, particularly on the public games endpoints.
- The API has no pagination on collection endpoints. For users with large collections this could become a performance concern.

8. Version Control Practices

The project follows a structured Git workflow throughout development. A develop branch serves as the integration branch, with feature branches created for each discrete unit of work and merged back via pull requests. The main branch is reserved for stable, submission-ready code only.

All commits follow conventional commit format (feat:, fix:, chore:, docs:, test:, refactor:) with scoped prefixes indicating the affected module. Pull requests use a consistent template including Summary, Changes, Implementation Notes, Testing checklist, and Notes sections. This approach was chosen to demonstrate professional version control discipline and to produce a commit history that clearly narrates the development progression.

9. Generative AI Declaration and Reflection

9.1 Tools Used

I used Claude (Anthropic) via claude.ai throughout this project. All conversations were conducted within a dedicated Claude Project, with the coursework brief, development conventions, and key architectural decisions stored as persistent context in the project instructions.

9.2 How GenAI Was Used

My use of Claude evolved across several distinct modes throughout the project, and I deliberately reflected on this evolution as part of the learning process.

In the **planning phase**, I used Claude to evaluate technology choices rather than simply accepting its first suggestion. I asked it to compare FastAPI, Django REST Framework, and Express across dimensions relevant to this specific project (auto-generated documentation, development speed, and framework maturity) and then made the final decision myself after reviewing the trade-offs. Similarly, I asked Claude to surface multiple Kaggle datasets for the board games domain and evaluated them against criteria such as column richness, file format simplicity, and licensing.

During **implementation**, Claude was used collaboratively rather than as a code generator. I would describe what I wanted to build, review the generated code critically, ask follow-up questions about specific design decisions (such as why static routes must precede dynamic ones in FastAPI), and adapt the output to fit my existing code. Debugging was kept as my own responsibility. I read error messages, understood the failure, and then used Claude to confirm my diagnosis or explore the fix.

One particularly effective **workflow** I followed was the use of Claude Projects with structured context. Rather than starting each session from scratch, I maintained the coursework brief, Git conventions, PR template, JWT auth flow, and testing conventions as persistent instructions. This significantly reduced repetition and kept the outputs consistent with the established conventions throughout the project. I also used the message editing feature to trim resolved debugging tangents from the chat history, keeping the conversation focused and preventing major context drift.

For **testing**, I used Claude to generate the initial test scaffolding based on the actual router files I uploaded, but reviewed each test for correctness and ran them myself. When test failures occurred, such as test ordering issues in the auth fixture, I diagnosed the root cause independently before asking Claude to confirm the fix. This maintained my genuine understanding of the codebase.

9.3 Critical Reflection

The biggest learning point from using GenAI on this project was not about any specific technical outcome, but about how to structure the interaction itself. Treating Claude as excellent mentor that is available 24/7 produced better outcomes than using it as a simple code generator. The quality of Claude's output was directly proportional to the specificity and context of my inputs, which reinforced the importance of precise technical communication.

I also observed that GenAI works best when the developer retains ownership of debugging and understanding. On several occasions I encountered errors that I could have simply pasted into Claude for a fix, but chose to diagnose first. This preserved the learning value of the exercise and ensured I could better explain every line of code in the oral examination.

10. References

- andrewmvd (2021). Board Games Dataset. Kaggle. https://www.kaggle.com/datasets/andrewmvd/board-games
- FastAPI Documentation. Tiangolo. https://fastapi.tiangolo.com
- SQLAlchemy Documentation. https://docs.sqlalchemy.org
- Pydantic Documentation. https://docs.pydantic.dev
- python-jose. https://python-jose.readthedocs.io
- passlib Documentation. https://passlib.readthedocs.io
- pytest Documentation. https://docs.pytest.org
- BoardGameGeek. https://boardgamegeek.com

Appendix A – GenAI Conversation Logs

The following screenshots document key interactions with Claude (Anthropic) throughout the development of BGG Vault API. AI was used for planning, technical decision-making, debugging, and code generation. All outputs were reviewed, tested, and adapted by the author.

Figure A1: Dataset selection – evaluating Kaggle options

Give me a more specific one for boardgames. Suggest datasets from Kaggle and Google Dataset Search, etc too

Great direction — board games actually have **excellent public datasets** and a rich domain for analytics. Let me search for what's available.

Searched the web >

Good, there's a rich ecosystem here. Let me now put together a concrete, specific recommendation.

Project: BoardGameGeek Analytics & Collection API

A personal board game collection tracker with analytics powered by real BGG data. The angle is: *"what should I play next, and why?"* — which is something you genuinely care about as a gamer.

Recommended Datasets (pick one)

Dataset	Source	Why it's good
Board Games Database from BoardGameGeek (threnjen)	Kaggle	Most comprehensive — includes mechanics, categories, ratings, weight, player counts
20,000 Boardgames Dataset	Kaggle	Large, clean CSV, easy to import
BoardGameGeek Reviews	Kaggle	Adds a review/ratings layer on top
BGG Dataset (IEEE DataPort)	IEEE DataPort	Academic-quality, well-documented, citable

Best choice for your project: the threnjen Kaggle dataset — it has fields like `mechanics`, `categories`, `min/max players`, `play_time`, `complexity_weight`, `avg_rating`, and `year_published`, which directly power interesting analytics endpoints.

Claude was used to compare available BoardGameGeek datasets on Kaggle, evaluating each against the project's data requirements. The andrewmvd/board-games dataset was manually selected based on this analysis.

Figure A2: Framework comparison – FastAPI vs Django vs Express

What's the ideal tech stack for this coursework? I'm thinking FastAPI bcs of its auto-generated API docs. What about other frameworks, like Django?

Good instinct on FastAPI. Let me break down the main options properly so you can make an informed choice and justify it in your technical report.

FastAPI vs Django REST Framework vs Express

FastAPI

Pros

- Auto-generates Swagger UI at `/docs` and Redoc at `/redoc` from your type annotations alone — zero extra documentation work, and it stays in sync with your code automatically
- Pydantic validation built in — request/response schemas are declared once and enforced everywhere
- Modern, async-first, increasingly the industry standard for pure Python APIs
- Lightweight — no ORM, no admin panel, no templating engine baggage you don't need

Cons

- Smaller ecosystem than Django
- Less opinionated, so you make more architectural decisions yourself

---

Django REST Framework (DRF)

Pros

- Mature, battle-tested, huge ecosystem
- Built-in admin panel, ORM, migrations out of the box

Cons

- Significantly more boilerplate for a pure API project
- Documentation requires extra setup (drf-spectacular or similar)
- Heavier than necessary when you don't need Django's full feature set

---

Node.js + Express

Pros

- Very flexible, large ecosystem

Cons

- No built-in validation or documentation generation
- Everything needs manual wiring

Recommendation

FastAPI is the right choice for this specific coursework for three reasons:

1. The brief requires API documentation as a deliverable — FastAPI gives you Swagger UI for free, which also doubles as an interactive demo for your oral exam
2. Pydantic schemas satisfy the brief's requirement for input validation and proper error codes with minimal extra code
3. It's easier to justify in your technical report — FastAPI is a deliberate modern choice over Django, which shows architectural awareness rather than just defaulting to the familiar

For the database, pair it with **SQLite via SQLAlchemy** — SQLite requires zero installation, lives in a single file, and is easy for examiners to run locally. SQLAlchemy gives you a clean ORM layer and means swapping to PostgreSQL later would only require a connection string change.

Want help planning the project structure and endpoint design next?

Claude compared three framework options – FastAPI, Django REST Framework, and Node.js Express – across validation, documentation generation, and setup overhead. FastAPI was selected for three reasons specific to this coursework: its automatic Swagger UI satisfies the documentation deliverable requirement, Pydantic schemas handle input validation natively, and it is a deliberate modern choice that demonstrates architectural awareness over defaulting to the more familiar Django.

Figure A3: Schema validation audit – identifying missing constraints

Any other possible practical constraints I'm missing from my implementation?

Good thing to audit before moving on. Here are the practical gaps worth fixing:

---

1. Username length and format — `app/schemas.py`

- `min_length=3` — prevents empty or single character usernames
- `max_length=30` — sensible cap
- `max_length=72` — bcrypt silently truncates passwords over 72 bytes, so capping it here makes the behaviour explicit rather than surprising
- Alphanumeric validator — prevents usernames like `"<script>"` or `"user name"` with spaces

---

2. Notes length — `app/schemas.py`

```python
class CollectionCreate(BaseModel):
    game_id: int
    status: str
    personal_rating: Optional[float] = Field(default=None, ge=1.0, le=10.0)
    play_count: int = Field(default=0, ge=0)
    notes: Optional[str] = Field(default=None, max_length=500)
```

Without this someone could POST a 10MB string into notes and it goes straight into your SQLite database.

---

3. Status as an Enum — `app/schemas.py`

Rather than validating status as a plain string in the router with `VALID_STATUSES`, make it an Enum so Pydantic rejects invalid values automatically and Swagger shows a dropdown:

Then remove the manual `VALID_STATUSES` check from `collection.py` entirely — it's now redundant.

---

4. Query param constraints on games router — `app/routers/games.py`

```python
players: Optional[int] = Query(default=None, ge=1, le=20)
```

Without this `?players=-5` or `?players=9999` would either return nothing silently or produce odd results.

---

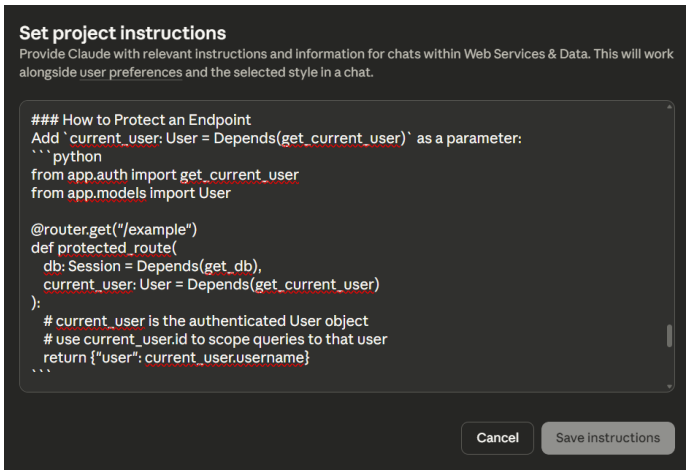
5. `game_id` must be positive — `app/schemas.py`

```python
class CollectionCreate(BaseModel):
    game_id: int = Field(gt=0)
    ...
```

Prevents `game_id: 0` or `game_id: -1` which would never match a real game but still hits your database.

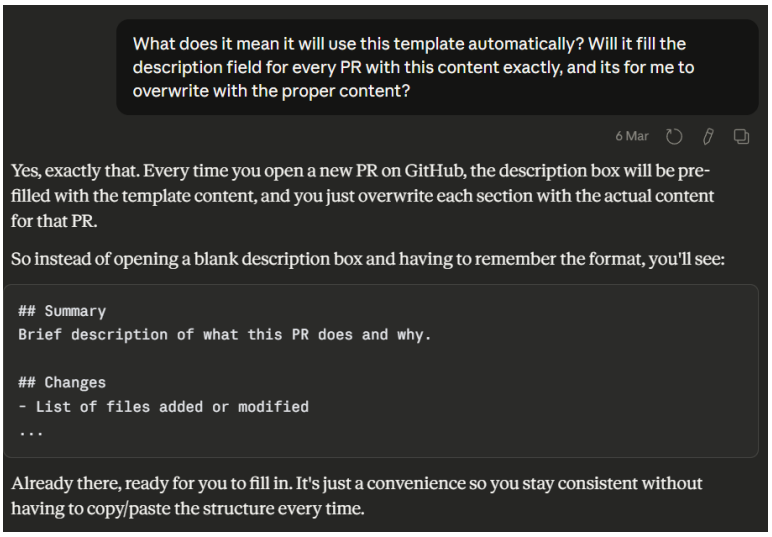
Claude reviewed the existing Pydantic schemas and identified missing field constraints such as rating bounds, play count minimums, and string length limits. These were incorporated into the final schemas to improve input validation and API robustness.

Figure A4: Claude Project context panel showing stored conventions



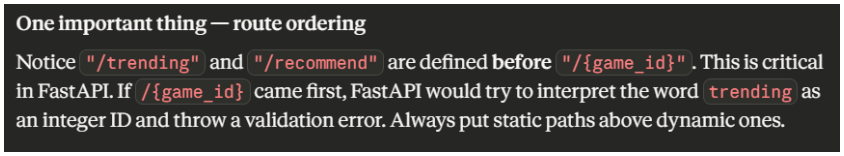
The Claude Project feature was used to persist development conventions – including commit format, branching strategy, and PR templates – across sessions. This ensured consistency throughout the development process without needing to re-establish context each conversation.

Figure A5: PR template discussion



Claude was used to establish a consistent PR description template following conventional commits format. The resulting template was saved as .gitub/PULL_REQUEST_TEMPLATE.md and applied across all feature branch merges into dev.

Figure A5: Route ordering concept explained



Claude explained why static routes must be defined before dynamic ones in FastAPI, preventing path segments like at `/trending` from being incorrectly parsed as integer IDs. This directly informed the structure of the games router.

Figure A6: Test fixture debugging

I got this error. From reading it I think the problem is that the user account used in testing doesn't exist in the test database when it runs. So need to add fixtures?

tests/test_auth.py::test_login_success FAILED
[100%]
========== FAILURES

Show more

23:30 2/2

You've diagnosed it correctly. The `test_login_success` test assumes `testuser` already exists, but it only gets created by the `registered_user` fixture in `conftest.py` — and that fixture only runs when a test actually requests it. Since `test_login_success` doesn't request it, the user doesn't exist yet.

The fix is simple — make `test_login_success` and the tests that depend on an existing user explicitly use the `registered_user` fixture:

`test_login_unknown_user`, `test_get_me_no_token` and `test_get_me_invalid_token` don't need it since they don't depend on a real user existing. `test_get_me_success` already uses `auth_headers` which chains from `registered_user` so that one is fine.

Claude assisted in diagnosing a test fixture issue, explaining the root cause and the correct approach to database session management in pytest. This accelerated debugging and deepened understanding of FastAPI's dependency injection system.